Python-Functions

Objective

- Defining a function
- Docstrings
- return statement
- Types of functions
- Function Arguments
- Anonymous functions

Defining a function

- In Python, a function is a group of related statements that performs a specific task.
- Functions help break our program into smaller and modular chunks. As our program grows larger and larger, functions make it more organized and manageable.
- It avoids repetition and makes the code reusable.

There are mainly two types of functions.

1. **User-define functions -** The user-defined functions are those define by the user to perform the specific task.

2. **Built-in functions** - The built-in functions are those functions that are pre-defined in Python. E.g. range() or print()

www.object.co.in

Advantage of Functions in Python

OBJECT™ TECHNOLOGIES

There are the following advantages of Python functions.

1. Using functions, we can avoid rewriting the same logic/code again and again in a program.
2. We can call Python functions multiple times in a program and anywhere in a program.
3. We can track a large Python program easily when it is divided into multiple functions.
4. Reusability is the main achievement of Python functions.

Syntax of Function

OBJECT™ TECHNOLOGIES

```
def function_name(parameters):
    """docstring"""
    statement(s)
    return expression
```

1. Keyword **def** that marks the start of the function header.
2. A **function name** to uniquely identify the function. Function naming follows the same rules of writing identifiers in Python.
3. **Parameters (arguments)** through which we pass values to a function. They are optional.
4. **A colon (:)** to mark the end of the function header.
5. Optional documentation string (**docstring**) to describe what the function does.
6. One or more valid python statements that make up the function body. Statements **must have the same indentation level** (usually 4 spaces).
7. An optional **return statement** to return a value from the function.

www.object.co.in

Example of a function

Example 1:

```
#function definition
def hello_world():
    print("hello world")

# function calling
hello_world()
```

Output:

hello world

Example2:

```
def greet(name):
    """
    This function greets to
    the person passed in as
    a parameter
    """
    print("Hello, " + name + ".
Good morning!")

greet('Ram')
```

Output:

Hello, Ram. Good morning!

www.object.co.in

Docstrings

- The first string after the function header is called the **docstring** and is short for documentation string. It is briefly used to explain what a function does.
- we have a docstring immediately below the function header.
- This string is available to us as the **__doc__** attribute of the function.

For example:

```
>>> print(greet.__doc__)
This function greets to the person passed in as a parameter
```


www.object.co.in

return statement

- **If the return statement has no expression or does not exist itself in the function then it returns the None object.**
- In Python we can **return multiple values.**

Syntax:

```
return [expression_list]
```


www.object.co.in

Example 1:

```
# Defining function
def sum():
    a = 10
    b = 20
    c = a+b
    return c


# calling sum() function in print statement
 print("The sum is:",sum())
```

Output:

The sum is: 30

Returning multiple values from function

OBJECT™ TECHNOLOGIES

Example:

```
def myFun(a,b):
    return b,a

a,b=input("Enter two no").split(" ")
print(a,b)

a,b=myFun(a,b)
print(a,b)
```

output:

```
Enter two no12 25
12 25
25 12
```

Example 2:

```
def test2():
    return 'abc', 100, [0, 1, 2]

a, b, c = test2()
print(a)
# Output: abc

print(b)
# Output: 100

print(c)
# Output: [0, 1, 2]
```

Types of functions

OBJECT™ TECHNOLOGIES

There are three types of functions in Python:

1. **Built-in functions :-**

   such as help() to ask for help, min() to get the minimum value, print() to print an object to the terminal,

2. **User-Defined Functions (UDFs):-**

   which are functions that users create to help them out;

3. **Anonymous functions:-**

   which are also called lambda functions because they are not declared with the standard def keyword.

Function Arguments

- The arguments are types of information which can be passed into the function.
- The arguments are specified in the parentheses.
- We can pass any number of arguments, but they must be separate them with a comma.

```
#Python function to calculate the sum of two variables
#defining the function
def sum (a,b):
    return a+b;

#taking values from the user
a = int(input("Enter a: "))
b = int(input("Enter b: "))

#printing the sum of a and b
print("Sum = ",sum(a,b))
```

Output:

```
Enter a: 10
Enter b: 20
Sum =  30
```


www.object.co.in

Function Arguments

There may be several types of arguments which can be passed at the time of function call.

1. Required arguments
2. Keyword arguments
3. Default arguments
4. Variable-length arguments

Required Arguments:

- Required arguments are the arguments passed to a function in correct positional order.
- The number of arguments in the function call should match exactly with the function definition.

```
# function calculate returns the sum of two arguments a and b
  def calculate(a,b):
   return a+b


calculate(10) # generates error as missing a required arguments b.
```

Output:

```
TypeError: calculate() missing 1 required positional argument: 'b'
```

Keyword arguments

- Python allows us to call the function with the keyword arguments.
- This kind of function call will enable us to pass the arguments in the random order.
- The name of the arguments is treated as the keywords and matched in the function calling and definition. If the same match is found, the values of the arguments are copied in the function definition.

Example 1

```
#function func is with the name and message as the keyword arguments
def func(name,message):
   print("printing the message with",name,"and ",message)

   #name and message is copied with the values John and hello respectively
   func(name = "John",message="hello")
```

Output:

```
printing the message with John and  hello
```


www.object.co.in

Providing the values in different order at the calling

#The function simple_interest(p, t, r) is called with the keyword arguments the order of arguments doesn't matter in this case .

```
def simple_interest(p,t,r):
    return (p*t*r)/100

print("Simple Interest: ",simple_interest(t=10,r=10,p=1900))
```

Output:
Simple Interest: 1900.0

If we provide the different name of arguments at the time of function call, an error will be thrown.

Default arguments:

- Python allows us to initialize the arguments at the function definition.
- If the value of any of the arguments is not provided at the time of function call, then that argument can be initialized with the value given in the definition even if the argument is not specified at the function call.

Example 1:

```
def printme(name,age=22):
   print("My name is",name,"and age is",age)
printme(name = "Ram")
printme(name = "Madhu",age=60)
```

Output:

```
My name is Ram and age is 22
My name is Madhu and age is 60
```


www.object.co.in

Variable-length arguments

- In large projects, sometimes we may not know the number of arguments to be passed in advance. In such cases, Python provides us the flexibility to offer the comma-separated values which are internally treated as tuples at the function call. By using the variable-length arguments, we can pass any number of arguments.
- However, at the function definition, we define the variable-length argument using the *args (star) as *<variable - name >.
- An asterisk (*) is placed before the variable name that holds the values of all nonkeyword variable arguments.
- This tuple remains empty if no additional arguments are specified during the function call.

Syntax for non-keyword variable arguments–

```
def functionname([formal_args,] *var_args_tuple ):
  "function_docstring"
  function_statements
  return [expression]
```


www.object.co.in

```
# Function definition is here
def printinfo( arg1, *vartuple ):
   "This prints a variable passed arguments"
   print (“Result is: “)
   print (arg1)
   for var in vartuple:
      print (var)
   return;

# Now you can call printinfo function
printinfo( 10 )
printinfo( 70, 60, 50 )
```

Output:

```
Result is:
10
Result is:
70
60
50
```

Anonymous functions

OBJECT™ TECHNOLOGIES

1. These functions are called anonymous because they are not declared in the standard manner by using the def keyword.
2. You can use the lambda keyword to create small anonymous functions.
3. Lambda forms can take any number of arguments but return just one value in the form of an expression. They cannot contain commands or multiple expressions.
4. An anonymous function cannot be a direct call to print because lambda requires an expression
5. Lambda functions have their own local namespace and cannot access variables other than those in their parameter list and those in the global namespace.
6. **A lambda function is a small anonymous function.**
7. **A lambda function can take any number of arguments, but can only have one expression.**

www.object.co.in

Syntax
lambda arguments : expression
The expression is executed and the result is returned:

Example
A lambda function that adds 10 to the number passed in as an argument, and print the result:

```
x = lambda a : a + 10
print(x(5))
```

Output:
15

Lambda functions can take any number of arguments:
Example
A lambda function that multiplies argument a with argument b and print the result:

```
x = lambda a, b : a * b
print(x(5, 6))
```

Output:
30

Summary

- Defining a function
- Docstrings
- return statement
- Types of functions
- Function Arguments
- Anonymous functions

Thank You